



x402-Meridian

Security Assessment

August 31, 2026 — Prepared by Chris

Chris

ra3964@nyu.edu

Table of Contents

Overview 3

Key Findings 3

01 — Executive Summary

Overview

Audit of x402-Meridian. This assessment was completed on August 31, 2026.

Key Findings

We produced 6 findings throughout this audit engagement. In particular, we identified a critical-risk vulnerability where Ethereum signatures are not bound to the expected permit data because the signed message is never compared with the permit hash, allowing previously observed signatures to be reused with attacker-controlled transfer parameters to authorize unauthorized token transfers (SP-XM-ADV-00). Also, fully consumed nonce bitmap accounts can be closed by any caller and subsequently reinitialized with all bits cleared, causing previously consumed or invalidated nonces to become reusable for replay attacks (SP-XM-ADV-01). Additionally, permit deadlines are included in signed data but are not enforced against the on-chain clock, allowing expired transfer and settlement authorizations to be executed indefinitely (SP-XM-ADV-02).

Furthermore, zero-value transfers are accepted after their unordered nonces have been irreversibly consumed, allowing valid permits to be invalidated without transferring any tokens (SP-XM-ADV-03). We also found that facilitators are required to fund nonce bitmap rent without receiving an on-chain fee, while sparse nonce selection can prevent those bitmap accounts from ever becoming closable and permanently lock facilitator-funded lamports (SP-XM-ADV-04). Additionally, unrestricted Token-2022 mints with permanent delegates can be used for settlement, allowing apparently finalized payments to be transferred away or burned by the mint authority after goods or services have been delivered (SP-XM-ADV-05).

02 — Scope

This audit was performed against commit `e0c2b8a` . Brief descriptions of the programs are as follows:

Name	Description
<code>permit2</code>	Solana program <code>permit2</code> under audit.
<code>x402</code>	Solana program <code>x402</code> under audit.

03 — Findings

Overall, we reported 6 findings.

Severity	Count
CRITICAL	1
HIGH	2
MEDIUM	1
LOW	2
INFO	0

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

ID	Severity	Description
SP-XM-ADV-00	CRITICAL	Signature verification checks only message length, allowing prior signatures to authorize arbitrary vault transfers.
SP-XM-ADV-01	HIGH	Unauthenticated bitmap closure allows zeroed PDA recreation, reviving consumed nonces and enabling repeated permit replay.
SP-XM-ADV-02	HIGH	Permit deadlines are signed but never checked, allowing stale authorizations to transfer subsequently deposited funds.
SP-XM-ADV-03	MEDIUM	Zero-value permit transfers consume the nonce without moving tokens, enabling payment denial and misleading execution events.
SP-XM-ADV-04	LOW	Facilitators receive no fee while funding sparse nonce bitmaps whose rent cannot be reclaimed, making micropayments loss-making.
SP-XM-ADV-05	LOW	Arbitrary Token-2022 mints let privileged mint authorities reverse settled payments or freeze vault funds.

Description

The `secp256k1` instruction payload is not compared with the computed `data_hash`. In `parse_secp256k1_instruction`, only the message length is validated, while `verify_eth_signature` checks only that the recovered `eth_address` matches. The native `secp256k1` program validates the signature against the message embedded in its own instruction and does not bind that message to the permit parameters.

Consequently, any previously published signature from the same address can be replayed before a new `permit_transfer_from`, `settle`, or `invalidate_unordered_nonces` instruction. An arbitrary token, amount, spender, nonce, deadline, witness, and recipient can be supplied while the old 32-byte signed message satisfies all implemented checks.

A fresh nonce allows the forged permit to proceed, while arbitrary nonce invalidations can permanently deny legitimate permits. All tokens held in the affected address's vaults can therefore be transferred on-chain to an unauthorized recipient.

```
// permit2/src/crypto.rs
fn parse_secp256k1_instruction(data: &[u8], self_index: u16, data_hash: &[u8; 32]) -
> Result<Secp256k1offsets> {
    // ...
    let message = data[message_offset..message_offset + message_size].to_vec();

    require!(
        data_hash.len() == message.len(),
        Permit2Error::InvalidSignature
    );
    // ...
}
```

Remediation

Replace the length-only check with

```
require!(offsets.message.as_slice() == data_hash.as_slice(), in
Permit2Error::InvalidSignature);
```

`verify_eth_signature`. Also parse and require the `signature_offset` and `recovery_id_offset` instruction indices to equal `self_index`.

Unauthorized Bitmap Closure Revives Consumed Nonces

HIGH

SP-XM-ADV-01

Description

`close_nonce_bitmap` can be invoked by any caller because `CloseNonceBitmap` contains no signer or Ethereum signature constraint. Once both bitmap halves equal `u128::MAX`, the deterministic PDA derived from `[NONCE_BITMAP_SEED, eth_address, word_pos]` may therefore be closed without authorization, including after all nonces have been consumed or invalidated.

```
// permit2/src/context.rs
pub struct CloseNonceBitmap<'info> {
    #[account(
        mut,
        seeds = [NONCE_BITMAP_SEED, eth_address.as_ref(), &word_pos.to_le_bytes()],
        bump = nonce_bitmap.bump,
        close = rent_payer,
    )]
    pub nonce_bitmap: Account<'info, NonceBitmap>,
    // ...
    #[account(mut, address = nonce_bitmap.rent_payer)]
    pub rent_payer: UncheckedAccount<'info>,
}
```

The closed PDA may subsequently be recreated by `PermitTransfer.nonce_bitmap` through `init_if_needed`, resetting `word_lo` and `word_hi` to zero. Because `use_unordered_nonce` only checks whether the selected bit is set, all 256 historical nonces become valid again. The bitmap may be filled, closed, and recreated repeatedly, allowing previously signed permits—including a `500_000_000` transfer—to be replayed until the vault's on-chain token balance is depleted.

Remediation

Require authorization by adding `instructions_sysvar` and `#[account(mut)] payer: Signer` to `CloseNonceBitmap`, then verify an Ethereum signature over

```
keccak256(&[DOMAIN_TAG, b"CloseNonceBitmap", eth_address.as_ref(), . Additionally,
&word_pos.to_le_bytes()])
```

prevent closed nonce words from being recreated with zeroed state by retaining the bitmap or recording a persistent tombstone or generation value.

Description

`permit.deadline` is authenticated by `hash_permit_transfer_from` and `hash_permit_witness_transfer_from`, but it is not enforced during execution. In `permit_transfer_from_common`, the requested amount, mint, Ethereum address, recipient, nonce, and signature are validated without comparing the deadline against `Clock::get()`. Similarly, `x402::settle` only verifies that the current timestamp is at least `witness.valid_after`; no upper time bound is applied.

Consequently, an unused signed permit may be retained indefinitely and submitted after the expected expiration time, including after the vault is subsequently funded or business conditions change. Waiting for expiration does not revoke the authorization;

`invalidate_unordered_nonces` must be called explicitly. A stale permit can therefore transfer up to `permit.permitted_amount` from the user's vault to the signed recipient on-chain.

```
// permit2/src/instructions.rs
pub fn permit_transfer_from_common(
    ctx: Context<PermitTransfer>,
    permit: PermitTransferFromArgs,
    transfer_details: SignatureTransferDetails,
    eth_address: [u8; 20],
    data_hash: [u8; 32],
) -> Result<()> {
    require!(
        transfer_details.requested_amount <= permit.permitted_amount,
        Permit2Error::InvalidAmount
    );
    // ...
```

Remediation

Add `require!((Clock::get()?.unix_timestamp as u64) <= permit.deadline, immediately Permit2Error::PermitExpired);`

after the requested-amount validation in `permit_transfer_from_common`. Ensure `Permit2Error::PermitExpired` is defined and returned for expired permits.

Zero-Value Transfers Permanently Consume Permit Nonces

MEDIUM

SP-XM-ADV-03

Description

Only an upper bound is enforced for `transfer_details.requested_amount`, allowing a value of `0` to satisfy `requested_amount ≤ permitted_amount`. In `permit_transfer_from_common`, `use_unordered_nonce(...)` is invoked unconditionally, while `transfer_checked` is executed only when the requested amount is greater than zero.

Consequently, a valid signed permit may be submitted with `requested_amount = 0`, permanently setting its nonce bit without transferring funds. `PermitTransferExecuted` is still emitted with `amount: 0`, which may cause off-chain facilitators to interpret the payment as completed. On-chain, the permit becomes unusable, the vault and recipient balances remain unchanged, and the intended payment cannot be settled.

```
// permit2/src/instructions.rs
pub fn permit_transfer_from_common(
    ctx: Context<PermitTransfer>,
    // ...
) → Result<()> {
    // ...
    use_unordered_nonce(
        &mut ctx.accounts.nonce_bitmap,
        // ...
    );
    // ...
    if transfer_details.requested_amount > 0 {
        token_interface::transfer_checked(
            // ...
        );
    }
}
```

Remediation

Add `require!(transfer_details.requested_amount > 0, Permit2Error::InvalidAmount);` at the beginning of `permit_transfer_from_common` before `use_unordered_nonce(...)` is invoked.

Fee-Free Settlement Locks Facilitator-Funded Bitmap Rent

LOW

SP-XM-ADV-04

Description

The facilitator is assigned as `fee_payer` for each `nonce_bitmap` created through `init_if_needed` and also pays the transaction fee. However, `settle` sets `requested_amount` to the full `permit.permitted_amount`, transferring the entire payment to the recipient without allocating any facilitator fee.

Each $8+93=101$ -byte bitmap locks $(128+101)*6960 = 1,593,840$ lamports ≈ 0.00159 SOL. Rent can be reclaimed through `close_nonce_bitmap` only when both `word_lo` and `word_hi` equal `u128::MAX`. By selecting sparse nonces from distinct 256-nonce words, a payer causes a new bitmap to be created for every settlement while preventing any bitmap from becoming fully consumed.

For 1000 payments of 0.001 USDC, approximately 1.594 SOL in rent and 0.005 SOL in transaction fees is imposed for only 1 USDC of payments, making settlement costs 35 350 times the payment value. Consequently, 1000 sparsely populated bitmap accounts remain on-chain and approximately \$350 @ \$220/SOL stays locked by the facilitator.

```
// x402/src/lib.rs
pub fn settle(
    ctx: Context<Settle>,
    permit: PermitTransferFromArgs,
    eth_address: [u8; 20],
    witness: Witness,
) -> Result<()> {
    // ...
    let transfer_details = SignatureTransferDetails {
        to: ctx.accounts.recipient_token_account.key(),
        requested_amount: permit.permitted_amount,
    };
    // ...
}
```

Remediation

Split `permit.permitted_amount` between the recipient and a facilitator fee sufficient to cover transaction and first-use bitmap costs, and add the required transfer to the facilitator. Permit `close_nonce_bitmap` to close partially consumed bitmaps when closure is authorized by a valid signature from the corresponding `eth_address`, allowing rent to be recovered safely.

Unrestricted Token-2022 Mints Allow Reversal of Settled Payments

LOW

SP-XM-ADV-05

Description

`init_vault` accepts any interface mint without parsing its Token-2022 extensions, while `settle` and `permit_transfer_from_common` enforce only that the supplied mint and token accounts match `permit.permitted_token` and `witness.to`. No protocol allowlist or merchant approval is required, and `recipient_token_account` may be created for the merchant by another party. Consequently, mints containing `PermanentDelegate`, `FreezeAuthority`, `TransferHook`, or `MintCloseAuthority` are accepted as settlement assets.

```
// permit2/src/lib.rs
pub fn init_vault(ctx: Context<InitVault>, eth_address: [u8; 20]) -> Result<()> {
    let vault = &mut ctx.accounts.vault;
    vault.eth_address = eth_address;
    vault.mint = ctx.accounts.mint.key();
    vault.bump = ctx.bumps.vault;
    vault.token_account = ctx.accounts.vault_token_account.key();
    Ok(())
}
```

A mint initialized through `InitializePermanentDelegate(attacker)` retains indefinite control over every account for that mint. After 1,000,000 units are deposited and settled, the merchant's balance may be transferred using `TransferChecked` or destroyed using `Burn` despite successful nonce consumption and emission of `Settled`; `FreezeAccount` may likewise immobilize `vault_token_account`. Thus, after goods are delivered, all 1,000,000 units may be removed and the merchant's on-chain balance reduced to zero while the settlement remains recorded as successful, making 100% of the payment reversible.

Remediation

Parse mint extension data in `init_vault` and reject `PermanentDelegate`, `TransferHook`, `MintCloseAuthority`, any non-`None` freeze authority, and other extensions granting post-settlement control with `Permit2Error::UnsupportedMint`. Enforce an explicit allowlist against `ctx.accounts.mint.key()` in `settle` before performing the CPI.